

MSXPi v1.6 - Legacy Board Support

How the v1.6 CPLD firmware, BIOS/MSX-DOS ROM and Pi server run on every earlier MSXPi board, what each board needs, and what was found while testing. The maintained reference is `hardware/CPLD_Project/LEGACY_BOARDS.md` in the repository; this document summarises it.

Summary

- All boards run the same v1.6 ROM and server. The I/O ports (\$56/\$57/\$5A) and the ready flag have not changed since v0.7 Rev.4.
- Only the CPLD pin map differs between boards, plus whether the PCB wires the CPLD to the cartridge /WAIT line. There is exactly one CPLD image per board.
- Pin maps and /WAIT wiring were checked against the PCB layout in each release branch.
- Tested on hardware: PCB v0.7 Rev.7 (v0.8.2 / v1.0) in a Canon V-25, MSX-DOS 1 booting from the Pi, and PCB V1.3 Rev.1. Not yet tested: PCB V1.1 Rev.1 and V1.2.1b.

What to flash or burn on each board

Releases	PCB	CPLD image (ID)	ROM: msxpibios.rom on	msxpi.ini extras
v0.8.2, v1.0	v0.7 Rev.7	v0.8.2/MSXPi_v1.6_polled.pof (\$10)	27C256 EPROM, burned off-board	GPIO_CS_SETUP_NS=1000
v1.1, v1.2.1b	V1.1 Rev.1, V1.2.1b	v1.1/MSXPi_v1.6_wait.pof (\$14)	AT28C256, flashed in circuit	V1.2.1b: RPI_SHUTDOWN=26
v1.3, v1.4, v1.5	V1.3 Rev.1	MSXPi_v1.6.pof (\$0E)	AT28C256, flashed in circuit	RPI_SHUTDOWN=26

- **ROM:** every board uses the same `target/msxpibios.rom`, the one the build script produces.
- **Server:** the same v1.6 server on every Pi. Pins from the `msxpi-Jumper*.ini` file matching the board (the older boards use `msxpi-JumperLeft.ini`); the extras column lists what to add. Settings left out stay unset.
- **CPLD images** are in `hardware/CPLD_Project/`.

PCB v0.7 Rev.7 (releases v0.8.2 and v1.0) - setup

EPROM jumpers

- **/OE on CS1**, so the ROM answers in page 1 (4000h-7FFFh) only.
- **A14 jumper on A15**, so page 1 reads the lower half of the 27C256, where the programmer puts the 16 KB `msxpibios.rom`. (With the jumper on A14 the MSX would read the empty upper half.)
- **Never CS12**. The ROM then also answers in page 2, the MSX finds its AB header twice and initialises it twice, and the second init chains EXTBIO to itself. MSX-DOS still boots and `msxarch` works, but `p`, `pcopy` and `pver` hang the machine with interrupts off (Caps Lock dead).

Pi settings (msxpi.ini)

- Pins: the Left set, GPIO 21/20/16/12/25 (msxpi-JumperLeft.ini).
- RPI_SHUTDOWN: leave unset. The board has no shutdown button; with the button enabled, noise on the unconnected GPIO 26 read as a press and rebooted the Pi in a loop.
- var GPIO_CS_SETUP_NS=1000: settling time between CS going low and the first SPI clock edge. Without it the native GPIO engine latched bytes that were still settling on this board's hand-wired connection (checksum errors, transfers stopping after 1 byte). A slower clock did not help; this delay did. Needs native/libmsxpi_gpio.so rebuilt on the Pi (native/build.sh).

Server-status LED

- Off while msxpi-server.py is not running (before it starts, after it stops, after shutdown).
- While the server is online: the original activity light - lit while the Pi is ready, dark while it works on a command or a byte moves, so it flickers during transfers.
- The server announces online/offline with one SCLK edge outside a transfer. Every other CPLD firmware ignores it (checked in the testbench against v1.3), so the same server runs on all boards.
- Known limitation: the LED can light again after the Pi has halted. This board has no pull-down on READY/SCLK (the newer PCBs have R8), so the floating lines can look like an "online" announce. A 10 kOhm pull-down on READY (and SCLK) fixes it.

msxpi.ini settings added in v1.6

Setting	Default	Meaning
var RPI_SHUTDOWN=26	unset: no button	GPIO of the shutdown/reboot button. Set only on PCBs that have one (V1.2.1b, V1.3 Rev.1). A press must stay low for 200 ms; under 3 s reboots, 3 s or more shuts down.
var GPIO_CS_SETUP_NS=1000	0	Settling time (ns) before the first SPI clock edge of each byte, native engine only. PCB v0.7 Rev.7 needs about 1000.

Existing Pis on PCB V1.2.1b or V1.3 Rev.1 must add var RPI_SHUTDOWN=26 after updating the server, or their shutdown button stops working.

Build IDs (\$57 bits 5-0) and pver.com

\$57: bit 7 = /WAIT mode read-back, bit 6 = reserved, always 0 (keeps the port below \$FE, the openMSX marker), bits 5-0 = build ID. The ID field was widened from 4 to 6 bits; every existing ID reads as before. An ID identifies a board/firmware variant, not a build: rebuilding or fixing a variant keeps its ID.

ID	Firmware
\$02-\$06	Prototypes and sample PCBs, original firmware
\$07	v0.7 Rev.4 / v0.8.2 original
\$09	v1.0 original
\$0A	v1.1 original
\$0B	v1.2 original
\$0C	v1.2.1b original

ID	Firmware
\$0D	v1.3 original (also shipped with v1.5)
\$0E	v1.6 main build, map C, /WAIT
\$10	v1.6 for PCB v0.7 Rev.7 (v0.8.2, v1.0), polled, server-status LED
\$11, \$12, \$13	retired, never reuse
\$14	v1.6 for PCB V1.1 Rev.1 / V1.2.1b (map B), /WAIT

Next free ID: \$15. `pver.com` (MSX-DOS) reads \$57 and prints the board, the CPLD build and whether /WAIT transfers are available, then the server version (as `p ver`). With no MSXPi present it says so and returns without contacting the server.

Problems found and fixed while testing on PCB v0.7 Rev.7

Symptom	Cause	Fix
Pi rebooted in a loop	Shutdown-button interrupt on the unconnected GPIO 26	Button only when RPI_SHUTDOWN is set; 200 ms press filter
MSX never sent a byte	Damaged GPIO 25 (READY) on the first Pi	Replaced the Pi
<code>p</code> , <code>pcopy</code> , <code>pver</code> hung the MSX, Caps Lock dead	EPROM /OE on CS12: ROM initialised twice, EXTPIO chained to itself	Jumper on CS1
Link-claim call made the Ethernet driver probe the Pi	UNAPI driver probed on routine 129	ROM: routine 129 no longer probes (probe waits for a real Ethernet call)
Checksum errors, transfers stopping after 1 byte	Native engine clocked the first edge as soon as CS went low	<code>GPIO_CS_SETUP_NS=1000</code> in <code>msxpi.ini</code>
LED back on after shutdown	Monitor restarted the server during shutdown; floating SCLK/MISO	Monitor exits on SIGTERM; no online announce while stopping; SCLK/MISO held low at exit. Remaining: floating lines after halt (pull-down needed)

If a rare disk error still appears in MSX-DOS, choosing Retry completes the operation: each retry reads the sector again with a fresh checksum.

Version 1.6 performance

What made v1.6 fast is mostly on the Pi side and in the Z80 transfer loops, and both apply equally to polled mode. /WAIT burst was never the main factor.

- **Per-byte time on the Pi dropped by roughly 50 to 100 times.** Up to v1.5 the server handled each byte in Python (200-600 us per byte). v1.6 has a native C engine (native/`libmsxpi_gpio.so`, one Python-to-C call per 256 bytes, 250 ns half-period): a byte takes a few microseconds. This is entirely on the Pi, so the v0.8.2 board gets the full benefit.
- **The Z80 polled loops were rewritten** as generated tight transfer loops (`asm-common/transport/payload.z80`). They work on every board.
- **Why /WAIT burst adds little on top:** in burst mode each INIR still stalls until the Pi has finished that byte, so burst and polled both run at roughly the Pi's per-byte speed. What is left is fixed cost per sector, the same in both modes: command and handshake, Python processing and file I/O, checksum and retry, MSX-DOS/FAT work.

MSXPi releases compared

Area	v0.8.2	v1.0	v1.1	v1.2.1b	v1.3 - v1.5	v1.6
PCB	v0.7 Rev.7	v0.7 Rev.7	V1.1 Rev.1	V1.2.1b	V1.3 Rev.1	all of these
CPLD pin map	A	A	B	B	C	per board
/WAIT wire	none (pin 8 = LED)	none (pin 8 = LED)	pin 28	pin 28	pin 37	used where wired
Original build ID (\$57)	\$07	\$09	\$0A	\$0C	\$0D	\$0E / \$10 / \$14
ROM chip	27C256 EPROM	27C256 EPROM	AT28C256	AT28C256	AT28C256 (PLCC)	as the board
Pi GPIO pins	21/20/16/12/25, fixed	same, fixed	msxpi.ini	msxpi.ini	msxpi.ini, button on 26	msxpi.ini; button opt-in
MSX-Pi protocol	SYN sync, CRC blocks	ping sync, 1 KB blocks	3-byte header, 256-byte blocks	as v1.1	as v1.1 (v1.5: handshake, checksum, 512-byte blocks)	as v1.5, plus /WAIT bursts

Every release uses the same CPLD part (EPM3064ALC44-10). The v1.6 builds use 45-53 of its 64 macrocells.

Old MSX software with the v1.6 server

Old ROM / tools	Effort	Why
v1.5	None	Same protocol
v1.1 - v1.3	Major	3-byte-header protocol with 256-byte blocks and different return codes
v1.0	Major	ping sync and 1 KB CRC blocks
v0.8.2	Major	SYN sync and its own command set

The practical route is to put the v1.6 ROM on every board (tables above) rather than keep four protocols in `msxpi-server.py`.

Raspberry Pi GPIO pins

File	CS	SCLK	MOSI	MISO	READY	Used for
msxpi-JumperLeft.ini	21	20	16	12	25	Default pins; all older boards
msxpi-JumperRight.ini	8	11	10	9	24	The Pi's hardware SPI pins
msxpi-JumperRight_PC BV1.1Rev.0.ini	8	11	10	9	25	PCB V1.1 Rev.0 (no READY jumper)

With no msxpi.ini the server uses the Left set. On PCB V1.2.1b and V1.3 Rev.1 the shutdown button is on GPIO 26 and must be enabled with `var RPI_SHUTDOWN=26`.